

역할 기반 인가와 작성자 권한 검증

학습 체크리스트 (1/2)

☐ `@AuthenticationPrincipal` 로 로그인 회원을 안전하게 받는 방법 습득

☐ `permitAll` 경로에서 `principal`이 `null` 이 되는 상황 이해

☐ 역할 기반 접근 제어(RBAC)의 구조와 `ROLE_` 접두사 규약 파악

☐ `authorizeHttpRequests` 로 경로별 인가 정책을 선언하는 방법 습득

학습 체크리스트 (2/2)

☐ 기본은 거부(Deny by Default) 원칙과 규칙 선언 순서 확인

☐ 게시판 작성자를 연관관계로 매핑하는 이유와 N+1 회피 방법 파악

☐ 서비스 계층에서 작성자 권한을 검증하는 위치와 방식 습득

☐ 화면 분기와 서버 인가의 역할 차이, 403·404 구분 기준 이해

학습 범위

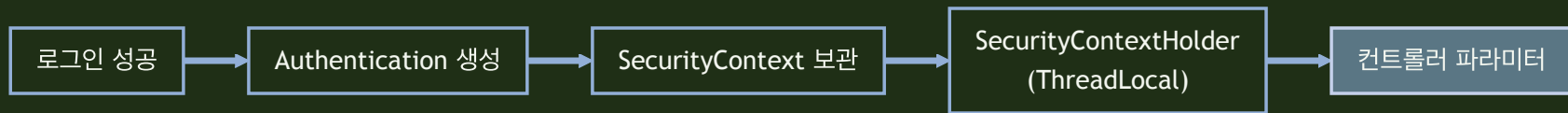
- 선행 조건: DB 기반 회원 관리 CRUD 완성
- 1. 회원 정보 확보: 컨트롤러에서 로그인 회원 정보 받아오기
- 2. RBAC 구성: 경로 단위 역할 기반 접근 제어 적용
- 3. 게시판 권한 검증: 게시판 CRUD와 작성자 권한 검증 구현

전체 진행 순서



로그인 회원 정보 확보

인증 정보가 컨트롤러에 도달하는 경로



인증 정보를 받는 네 가지 방법

방법	코드	특징
SecurityContextHolder 직접 조회	<code>getContext().getAuthentication()</code>	어디서든 가능, 캐스팅 필요
컨트롤러 파라미터	<code>Authentication authentication</code>	Spring MVC가 자동 주입
Principal 파라미터	<code>Principal principal</code>	서블릿 표준 타입, 캐스팅 필요
어노테이션 주입	<code>@AuthenticationPrincipal CustomUserDetails user</code>	캐스팅 없이 바로 사용

- 본 강의에서는 `@AuthenticationPrincipal` 을 기본 방식으로 사용합니다

로그인 회원을 파라미터로 받기

```
1 @GetMapping("/user-accounts/me")
2 String me(@AuthenticationPrincipal CustomUserDetails user, Model model) {
3     model.addAttribute("nickname", user.getNickname());
4     return "user-accounts/me";
5 }
```

@AuthenticationPrincipal이 대신 해주는 일

- 처리 주체: `AuthenticationPrincipalArgumentResolver`가 내부에서 작업
- 내부 동작: `getContext().getAuthentication().getPrincipal()` 호출 후 캐스팅
- 결과: 컨트롤러 코드에는 캐스팅 구문이 전혀 드러나지 않음
- 추가 이점: `CustomUserDetails`가 id·닉네임을 이미 보유해 재조회 불필요

permitAll 경로에서의 null 주입 주의점

- 익명 사용자의 정체: principal이 문자열 `"anonymousUser"` (`AnonymousAuthenticationToken`)
- 캐스팅 실패: `CustomUserDetails` 로 캐스팅할 수 없는 타입
- 결과: 예외가 아니라 파라미터에 `null` 이 조용히 주입됨
- 대응: `permitAll` 경로의 컨트롤러는 null 검사를 반드시 수행

로그인 여부와 무관하게 열리는 게시물 목록

```
1 @GetMapping("/boards")
2 String list(@AuthenticationPrincipal CustomUserDetails user, Model model) {
3     model.addAttribute("boards", boardService.findAll());
4     model.addAttribute("isLoggedIn", user != null);
5     return "boards/list";
6 }
```

필요한 필드 하나만 꺼내 받기

```
1 @GetMapping("/boards/new")
2 String form(@AuthenticationPrincipal(expression = "id") Long userAccountId,
3             Model model) {
4     model.addAttribute("writerId", userAccountId);
5     return "boards/form";
6 }
```

화면에서 로그인 상태를 다루는 준비

- 의존성 추가: `thymeleaf-extras-springsecurity6` 라이브러리 도입
- 네임스페이스 선언: html 루트 태그에 `xmlns:sec` 선언 필요
- 노출 조건 지정: `sec:authorize` 로 로그인 여부에 따른 노출 제어
- 로그인 아이디 출력: `sec:authentication="name"` 으로 화면에 표시

로그인 여부에 따라 갈리는 내비게이션

```
1 <nav>
2   <a sec:authorize="isAnonymous()" href="/login">로그인</a>
3   <a sec:authorize="isAuthenticated()" href="/user-accounts/me">
4     <span sec:authentication="name">아이디</span>님
5   </a>
6   <form sec:authorize="isAuthenticated()" action="/logout" method="post">
7     <button type="submit">로그아웃</button>
8   </form>
9 </nav>
```

화면 분기는 보안이 아니다

- 우회 가능성: UI에서 메뉴를 숨기더라도 URL 직접 입력 시 서버에 도달
- `sec:authorize`의 역할: 접근 차단이 아닌 UX 정리 수단일 뿐
- 실제 차단 지점: `authorizeHttpRequests` 경로 정책과 서비스 계층 검사
- 원칙: 화면 제어는 UX 용도일 뿐, 실제 접근 차단은 서버가 담당

경로 단위 인가 (RBAC)

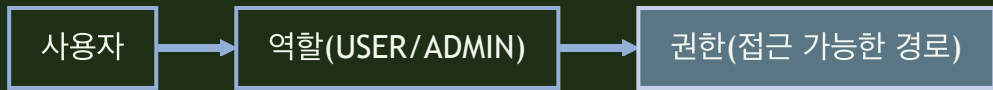
RBAC란 무엇인가

역할 기반 접근 제어 (RBAC)

권한을 사용자에게 직접 붙이지 않고 역할이라는 중간 계층을 두는 모델

- **이중 관계:** 사용자↔역할, 역할↔권한 두 관계로 분리
- **직무 변경 대응:** 직무가 바뀌면 역할만 교체하면 됨
- **관리 비용 절감:** 사용자가 늘어나도 역할 수는 그대로

역할이라는 중간 계층



역할과 권한은 같은 타입이다

- 역할 전용 타입 없음: Spring Security에는 역할만의 별도 타입이 없음
- 공통 표현: 역할과 권한 모두 `GrantedAuthority`로 표현
- 관례상 구분: `ROLE_` 접두사가 붙으면 관례상 역할로 취급
- 내부 위임: `hasRole`은 접두사를 붙여 `hasAuthority`에 위임

hasRole과 hasAuthority 비교

표현식	실제 비교하는 권한 문자열
<code>hasRole("ADMIN")</code>	<code>ROLE_ADMIN</code>
<code>hasAuthority("ROLE_ADMIN")</code>	<code>ROLE_ADMIN</code>
<code>hasAuthority("boards:write")</code>	<code>boards:write</code>

- **선택 기준:** 결과는 동일하므로 의도를 드러내는 쪽을 선택
- **설계 배경:** 앞서 `Role.key()` 가 `"ROLE_" + name()` 을 반환한 이유

역할(Role) 구조 설계

- 단순화: 역할은 `USER`, `ADMIN` 두 개로 단순화
- 기본 부여: 가입 시 기본으로 `USER` 역할 부여
- 관리자 전용: `ADMIN`은 관리자 전용 경로 접근에만 사용
- 범위 제한: 세밀한 권한 문자열 설계는 이번 범위 밖

경로별 인가 정책

경로	메서드	정책
<code>/css/**</code> , <code>/js/**</code>	ALL	<code>permitAll()</code>
<code>/</code> , <code>/login</code> , <code>/signup</code> , <code>/error/**</code>	ALL	<code>permitAll()</code>
<code>/boards</code> , <code>/boards/{id}</code>	GET	<code>permitAll()</code>
<code>/admin/**</code>	ALL	<code>hasRole("ADMIN")</code>
<code>/user-accounts/**</code>	ALL	<code>authenticated()</code>
그 외 모든 요청	ALL	<code>authenticated()</code>

경로 규칙을 선언하는 필터 체인

```
1 http.authorizeHttpRequests(auth → auth
2     .requestMatchers("/css/**", "/js/**").permitAll()
3     .requestMatchers("/", "/login", "/signup", "/error/**").permitAll()
4     .requestMatchers(HttpMethod.GET, "/boards", "/boards/{id}").permitAll()
5     .requestMatchers("/admin/**").hasRole("ADMIN")
6     .requestMatchers("/user-accounts/**").authenticated()
7     .anyRequest().authenticated());
```


로그인·로그아웃 경로 설정

```
1 http.formLogin(form → form
2     .loginPage("/login")
3     .loginProcessingUrl("/login")
4     .defaultSuccessUrl("/boards")
5     .failureUrl("/login?error")
6     .permitAll())
7 .logout(logout → logout
8     .logoutUrl("/logout")
9     .logoutSuccessUrl("/login?logout"));
```

규칙은 구체적인 것부터, anyRequest는 마지막

- 순차 평가: 규칙은 선언한 순서대로 위에서부터 평가됨
- 배치 원칙: 구체적 경로를 먼저, 넓은 패턴을 뒤에 배치
- 마지막 위치: `anyRequest()` 는 반드시 맨 마지막에 위치
- 순서 사고: 순서가 뒤바뀌면 뒤쪽 규칙이 무시됨

기본은 거부, 필요한 것만 허용

- Deny by Default: 명시하지 않은 경로는 기본적으로 거부
- 안전망 역할: `anyRequest().authenticated()`가 최종 안전망
- 자동 보호: 새로 추가된 경로도 자동으로 보호 대상이 됨
- 누락 방지: 인가 설정 누락으로 인한 보안 취약점을 사전에 예방

인가 정책을 한곳에 모으는 이유

- 파악 곤란: 정책이 흩어지면 어떤 경로가 보호되는지 알기 어려움
- 누락 위험: 여러 곳에 흩어질수록 누락 위험이 커짐
- 중앙 집중: 이번 표준은 `SecurityFilterChain` 에서 중앙 관리
- 범위 제외: `@PreAuthorize` 메서드 시큐리티는 개념만 확인, 이번 범위에서는 미사용

게시판 CRUD와 작성자 매핑

작성자를 이름 문자열로 두지 않는 이유

- 닉네임 변경 문제: `String authorName` 은 닉네임 변경 시 과거 글 표시가 어긋남
- 식별 불가: 회원 식별이 불가능해 작성자 검증 자체가 불가
- 연관관계 매핑: `UserAccount author` 연관관계로 매핑
- 비교 기준: 작성자 검증은 항상 작성자 id 기준

Board 엔티티의 필드 구성

```
1  @Entity @Getter
2  public class Board extends BaseTimeEntity {
3      @Id @GeneratedValue
4      private Long id;
5      private String title;
6      private String content;
7      @ManyToOne(fetch = FetchType.LAZY)
8      private UserAccount author;
9  }
```

글 작성·수정과 작성자 검증 메서드

```
1  public static Board write(String title, String content, UserAccount author) {
2      return new Board(title, content, author);
3  }
4  public void update(String title, String content) {
5      this.title = title;
6      this.content = content;
7  }
8  public boolean isAuthor(Long userAccountId) {
9      return author.getId().equals(userAccountId);
10 }
```


지연 로딩과 목록 화면의 N+1

- 지연 로딩: `FetchType.LAZY`로 작성자 즉시 로딩을 방지
- 추가 조회 발생: 목록에서 닉네임을 쓰면 한 건씩 추가 조회
- N+1 문제: 100건 목록이면 쿼리가 101번 실행됨
- 해결 방향: 조회 시점에 작성자까지 함께 가져와야 함

작성자를 한 번에 가져오는 조회 메서드

```
1 public interface BoardRepository extends JpaRepository<Board, Long> {  
2  
3     @EntityGraph(attributePaths = "author")  
4     List<Board> findAllByOrderByCreatedAtDesc();  
5  
6     @Query("select b from Board b join fetch b.author where b.id = :id")  
7     Optional<Board> findWithAuthorById(@Param("id") Long id);  
8 }
```

글 작성 컨트롤러가 하는 일

```
1  @PostMapping("/boards")
2  public String create(@Validated @ModelAttribute BoardForm form,
3                      @AuthenticationPrincipal CustomUserDetails user) {
4      boardService.create(form.getTitle(), form.getContent(), user.getId());
5      return "redirect:/boards";
6  }
```

폼에 작성자 id를 두지 않는 이유

- 조작 가능: hidden `authorId` 는 클라이언트가 임의로 조작 가능
- 위조 위험: 남의 이름으로 글이 등록될 수 있음
- 서버 확인 원칙: 작성자는 항상 서버가 인증 컨텍스트에서 확인
- 경로 차단: 클라이언트 값이 끼어들 자리를 아예 만들지 않음

글을 저장하는 서비스

```
1  @Transactional
2  public void create(String title, String content, Long authorId) {
3      UserAccount author = userAccountRepository.getReferenceById(authorId);
4      Board board = Board.write(title, content, author);
5      boardRepository.save(board);
6  }
```

getReferenceById로 불필요한 DB 조회 최소화

- 프록시만 생성: 실제 조회 없이 프록시만 생성해 연관관계 연결
- 이미 검증된 값: `authorId`는 이미 검증된 로그인 회원 id
- 재조회 불필요: 존재 확인 쿼리를 다시 보낼 필요 없음
- 적용 범위: 연관관계만 걸 때 적합한 방식

목록과 상세는 익명 방문자도 본다

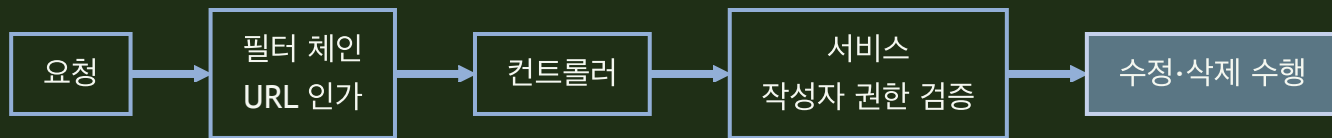
- 공개 경로: `GET /boards`, `/boards/{id}` 는 `permitAll`
- null 주입: 이때 `user` 는 `null` 로 주입됨
- 양쪽 분기 필요: 컨트롤러와 화면 양쪽에서 분기 필요
- 정상 경로: 여기서는 `user == null` 이 정상 경로

작성자 권한 검증

인증 정보만으로 인가를 처리할 수 없는 이유

- 다른 조건: "로그인했다"와 "이 글을 쓴 본인이다"는 다른 조건
- 판단 불가: URL 패턴만으로는 작성자 권한을 판단할 수 없음
- DB 조회 필요: DB에서 글을 읽어 작성자 id를 비교해야 함
- 필터 체인의 한계: 데이터에 종속된 규칙은 필터 체인이 처리할 수 없음

인가 판단이 나뉘는 두 지점



수정 요청의 조회·검사·변경

```
1  @Transactional
2  public void update(Long boardId, Long userAccountId, String title, String content) {
3      Board board = boardRepository.findById(boardId)
4          .orElseThrow(() → new NoSuchElementException("게시글 없음"));
5      if (!board.isAuthor(userAccountId)) {
6          throw new AccessDeniedException("작성자만 수정 가능");
7      }
8      board.update(title, content);
9  }
```

삭제 요청의 작성자 권한 검증

```
1  @Transactional
2  public void delete(Long boardId, Long userAccountId) {
3      Board board = boardRepository.findById(boardId)
4          .orElseThrow(() -> new NoSuchElementException("게시글 없음"));
5      if (!board.isAuthor(userAccountId)) {
6          throw new AccessDeniedException("작성자만 삭제 가능");
7      }
8      boardRepository.delete(board);
9  }
```

setter 대신 update 메서드를 두는 이유

- 무분별한 변경: setter가 열리면 어디서든 필드를 개별 변경 가능
- 통제 어려움: 의도치 않은 변경을 막기 어려움
- 의도 노출: `update( ... )` 는 변경 범위와 의도를 코드에 드러냄
- 동일한 이유: `isAuthor( ... )` 도 같은 이유로 엔티티 안에 배치

버튼을 작성자에게만 보여주기

```
1 <a th:href="@{/boards/new}" sec:authorize="isAuthenticated()">글쓰기</a>
2
3 <div th:if="${canEdit}">
4     <a th:href="@{/boards/{id}/edit(id=${board.id})}">수정</a>
5     <form th:action="@{/boards/{id}/delete(id=${board.id})}" method="post">
6         <button type="submit">삭제</button>
7     </form>
8 </div>
```

canEdit을 컨트롤러가 계산하는 이유

- 속성 접근 오류: 익명 사용자는 `#authentication.principal` 이 문자열이라 `id` 접근 시 오류
- 사전 판단: 컨트롤러가 `user == null` 까지 감안해 미리 판단
- UX일 뿐: 화면 분기는 사용성 개선일 뿐
- 실제 차단: 실제 차단은 서버의 작성자 권한 검증이 담당

삭제는 POST 폼으로

- CSRF 토큰: `th:action` + `method="post"`로 작성해야 CSRF 토큰이 함께 실림
- 링크의 한계: `<a href>` 링크는 토큰이 없어 403 응답
- GET 규약 위반: GET으로 삭제하면 부수효과 없는 GET 규약을 위반함

403과 404를 가르는 기준

상황	응답
권한 없음	403
자원이 없음	404
존재 자체를 감춰야 하는 자원	404로 통일

- 연결 구조: `AccessDeniedException` 은 `AccessDeniedHandler` 를 거쳐 `templates/error/403.html` 로 이어짐

인가 책임 분담 정리

항목	어디에 구현	왜 필요한가
로그인 회원 주입	@AuthenticationPrincipal	요청자 신원을 신뢰 가능한 방식으로 확보
필터 체인 인가	Security Config permitAll / authenticated	URL 패턴 단위 접근을 요청 초입에서 차단
작성자 권한 검증	서비스 계층 isAuthor(...)	데이터에 종속된 규칙은 DB 조회 후에만 판단 가능
화면 분기	컨트롤러·템플릿 canEdit	불필요한 버튼 노출 방지, 실제 차단은 아님

학습 요약 (1/2)

- **신원 확보:** `@AuthenticationPrincipal`로 로그인 회원을 받고 `null` 가능성을 항상 고려
- **RBAC:** 역할이라는 중간 계층으로 사용자와 권한을 분리, `ROLE_` 접두사는 `key()`에서만
- **중앙 집중 인가:** 경로 단위 정책은 `SecurityFilterChain` 한곳에 모아 선언
- **선언 순서:** 구체적 경로를 먼저, `anyRequest().authenticated()`는 언제나 마지막

학습 요약 (2/2)

- 작성자 매핑: 이름 문자열이 아닌 `UserAccount` 연관관계, 목록은 fetch join으로 N+1 회피
- 작성자 권한 검증: 데이터에 종속된 규칙은 서비스 트랜잭션 안에서 `isAuthor( ... )` 로 판단
- 신뢰 경계: 작성자는 폼 값이 아니라 언제나 인증 컨텍스트에서 확보
- 화면과 서버: `sec:authorize` · `canEdit` 은 UX 정리일 뿐, 차단은 항상 서버가 담당